

# AlpacaAgent in practice

## Use cases, agent prompts, and review templates

Worked workflows for learning the system, interpreting research, investigating decisions, and running a disciplined paper experiment.

### DETAILED WINDOWS EDITION

Based on the local implementation, with corrected commands, practical examples, and explicit operational boundaries.

Prepared 18 September 2026

Source revision 235b087

Companion volumes: handbook, use cases, and desk reference

# Contents

Choose a chapter below. PDF bookmarks and contents links navigate directly to each chapter.

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1. Get better answers from the agents                             | 3  |
| 2. Use case: understand the workspace in ten minutes              | 4  |
| 3. Use case: get a market briefing without mixing it with signals | 5  |
| 4. Use case: compare research without fooling yourself            | 6  |
| 5. Use case: investigate why there are no trades                  | 7  |
| 6. Use case: inspect a candidate from idea to refusal             | 8  |
| 7. Use case: understand a held position and its exit              | 9  |
| 8. Use case: pause, leave, or recover after an interruption       | 10 |
| 9. Use case: manage generative chat efficiently                   | 11 |
| 10. Templates for disciplined operation                           | 12 |
| 11. Weekly review and progression worksheet                       | 13 |

# 1. Get better answers from the agents

This volume accompanies the operator's handbook. It describes source revision 235b087, reviewed September 18, 2026. Prompts below are questions for the application; they do not create new capabilities. Examples containing numbers are synthetic unless explicitly labeled otherwise.

## First identify which chat you are using

Static dashboard chat is a deterministic keyword router. It chooses a canned explanation based on the embedded snapshot. It does not reason through a long prompt, remember a research plan, or execute a command. Use short topic questions and the suggested chips.

Served non-generative chat uses the same type of router but rereads current local records for each request. The exception is Houston's exact reconcile message, which requests a dry diagnostic cycle. "Please reconcile everything and then trade" is not the exact diagnostic command and cannot trade.

Generative chat is optional in served mode when ANTHROPIC\_API\_KEY is available at launch. It can use four local tools and two market-data tools: `read_snapshot`, `read_backtest`, `read_cycles`, `explain_rule`, `read_market`, and `read_news`. Market tools require the server's Massive/Polygon credentials and suitable access. They retrieve stock snapshots and headlines, not live option chains.

## A useful prompt has four parts

Name the question, identify the evidence, specify the output, and require uncertainty to remain visible. For example:

Astra: Read the latest local snapshot and recent cycles. List the three most important blockers, the source and timestamp for each, and the next diagnostic step. Keep missing records separate from confirmed zero values.

Keep prompts focused. The current generative implementation sends at most 800 characters of user text per question and retains a bounded per-agent history of 20 user/assistant messages. Long pasted reports are a poor interface; ask the tools to read their supported records. Restate key context when switching agents.

## What to look for in an answer

A good answer identifies the source, date, missing values, and whether it is describing local records or newly fetched market information. A fluent answer without record references is not a verified state check. If a response falls back to rule-based mode, the source text should explain that. Reformulating the same question will not repair an unavailable provider or missing data entitlement.

Do not paste credentials into chat. In generative mode, your message, bounded conversation history, and sanitized tool outputs can be sent to the configured Anthropic API. Local hosting does not mean every answer is computed entirely offline.

## 2. Use case: understand the workspace in ten minutes

### Goal and prerequisites

You want to understand what the system last did without starting paper trading. You need the project installed and whatever local runtime records already exist. Broker and market-data credentials are not necessary merely to open the workspace.

### Procedure

1. Run `python -m alpaca_agents.dashboard` and open `runtime/dashboard.html`, or start `controller --serve` and open its printed URL.
2. Check the rendered timestamp and latest cycle timestamp. They answer different questions: when the page was created, and when the controller last recorded work.
3. Look for missing/unreadable record warnings. An unknown ledger prevents conclusions about an empty account.
4. Read the control mode, open positions, and outstanding intents. DISABLED with a held position deserves immediate attention because automatic exits are disabled too.
5. Read recent reconciliation reasons and notifications before interpreting research or scanner cards.
6. Write a three-line summary: confirmed facts, unknown facts, and the next useful diagnostic.

### Prompts

Rule-based: "Give me a briefing." Then ask separately: "What are the blockers?", "What positions are recorded?", and "What do the controls mean?"

Generative:

Astra: Summarize the saved workspace state. Show control mode, latest cycle time, reconciliation outcome, held positions, outstanding intents, and missing records. Do not infer a running controller from a recent timestamp.

Houston: Explain the outstanding intents in the local snapshot. Distinguish reservations from claimed orders and say what evidence is missing before any operational decision.

### Example interpretation

Suppose the page was rendered at 14:00 UTC, the last cycle finished at 12:00 UTC, and the mode is DISABLED. You know the page was built recently from two-hour-old cycle evidence. You do not know that the broker was checked at 14:00 or that a controller is currently running. If you need broker state now, a fresh diagnostic is the next useful action.

### Completion criterion

You can name the files or timestamps supporting each statement. You have not equated a polished overview, a successful browser connection, or an agent's confident tone with fresh trading readiness.

## 3. Use case: get a market briefing without mixing it with signals

### Goal and prerequisites

You want context on the current market and a separate explanation of what AlpacaAgent did. Use served generative mode with an Anthropic key and suitable Massive data access. Static or deterministic chat cannot fetch a fresh market briefing.

### Recommended prompt sequence

Star: Use read\_market for SPY, QQQ, IWM, and DIA and read\_news for recent headlines. Give each snapshot's timestamp, the reported moves, and attributed headlines. Separate these observations from the system's saved scanner results. Do not turn a headline into a trade signal.

Astra: Now read recent cycles and explain what the system actually did today. Separate successful checks, missing data, skipped candidates, and submitted-order records. Identify the latest timestamp for each claim.

### How to evaluate the response

Stock snapshots establish stock-market observations only. They do not show that the scanner found an eligible 30-45 DTE contract under the risk cap. Headlines are third-party reports; a short provider description is not an independently verified explanation for a price move.

If one symbol has no available snapshot, the right result is a missing-data statement. If market data is unavailable entirely, ask for an explanation of saved local records or a general concept instead. Do not prompt the model to guess today's price from memory.

### Follow-up questions that help

Nova: Explain the difference between an underlying stock move, an option premium move, and an underlying backtest R-multiple. Use hypothetical numbers and label them as hypothetical.

Star: Which parts of this briefing are retrieved data, which are attributed news, and which are interpretation? What option-specific information would still be needed to evaluate a candidate?

### What this workflow cannot do

The chat tool does not fetch a live option chain or create a trade authorization. A report that "QQQ rose" does not mean trend\_directional fired, the contract was liquid, Moon approved it, or Houston sent an order. Follow the evidence from each layer instead of collapsing the story into one bullish or bearish conclusion.

### Completion criterion

You have a timestamped briefing and a separate operational summary. You can explain what is known about the market without treating it as an instruction to enable or submit a trade.

## 4. Use case: compare research without fooling yourself

### Goal

Compare the same playbook across symbols and determine whether the evidence is worth further testing. Keep data dates, code revision, and modeling assumptions consistent. Use the cached-bars workflow from Volume 1 to make comparisons reproducible.

### A synthetic comparison

| Measure               | Sample A | Sample B |
|-----------------------|----------|----------|
| Resolved observations | 40       | 38       |
| Mean R                | +0.25    | +0.90    |
| Median R              | +0.15    | -0.35    |
| Profit factor         | 1.45     | 2.20     |
| Worst loss R          | -1.80    | -7.50    |

Sample B has the larger mean but a negative typical observation and a much worse downside result. It deserves an outlier and concentration review, not automatic selection. Sample A is more internally consistent on these few measures, but neither table models option returns or proves a persistent edge.

### Procedure

1. Check `first_bar`, `last_bar`, `bars`, `source`, and `generated_at`. Exclude accidental comparisons of different periods.
2. Read `resolved`, `no_fill`, `no_trade`, and `unresolved` categories. A small number of resolved trades after many signals is informative.
3. Compare mean and median together. Inspect the largest winners and losers in the trades array.
4. Read `win_rate` separately from exits. A stop exit can be profitable; a timeout can be a win or loss.
5. Split your review by period or market regime where you can do so without repeatedly choosing the most flattering window.
6. Decide whether to reject, retain as inconclusive, or study in shadow mode. Record why before changing parameters.

### Generative prompts

Star: Read the QQQ backtest. Report the data window, resolved count, mean R, median R, profit factor, worst loss R, `no_trade` count, and model caveats. Explain what conclusions remain unsupported.

Moon: Explain why a positive underlying expectancy does not establish option profitability. Relate the missing option-price model to the system's actual risk and contract filters.

The chat tool exposes a report summary; inspect the JSON locally for trade-level calculations. Finish with a decision that includes favorable and unfavorable evidence, sample limitations, and the exact report. Do not convert R to dollars or select solely on mean R.

## 5. Use case: investigate why there are no trades

### Goal

Identify the first missing condition without changing it simply to produce activity. "No trades" can be the correct result at many layers.

### Follow this diagnostic sequence

1. Is the controller actually running? Check its terminal and latest cycle time. Opening the dashboard does not start it.
2. Did reconciliation pass? If not, read every reason and stop investigating hypothetical entries until state is understood.
3. What is the control mode? DISABLED and EXITS\_ONLY skip the controller's entry stage. Use the standalone scanner for disabled shadow research.
4. Did the scanner obtain valid snapshots? HTTP errors, missing Greeks, stale quotes, and malformed data are not evidence of absent setups.
5. Did a signal pass? No trend, excessive chase, tight stop, or weak reward:risk can correctly eliminate it.
6. Was a contract eligible and affordable? DTE, delta, open interest, bid/ask width, and total premium can remove an otherwise valid signal.
7. Was the matching playbook approved and enabled at launch? A marker by itself is insufficient.
8. Did Moon or the journal reject the candidate? Look for the exact reason, not a paraphrase such as "risk issue."
9. Was --submit present? Without it, an ordinary enabled dry cycle can reserve but will not submit.
10. Was an order submitted but unfilled? That is an execution result, not an absence of proposals.

### Prompts

Astra: Using the latest cycles and snapshot, identify the earliest confirmed blocker in the path from reconciliation to submission. Say which later stages have no evidence because the cycle stopped early.

Star: Explain the recorded scan errors and skips. Distinguish invalid inputs from valid no-signal outcomes. Do not suggest loosening a filter simply to get a candidate.

Moon: Explain the exact rejection reason in the latest entry record and the invariant it protects. If the record is unavailable, say which source is missing.

### Worked example

Assume reconciliation passed and control is DISABLED. The correct controller record can say entries were skipped for that mode, with no scan stage. That is expected. If a separately run shadow scan returns three HTTP 403 errors, the next task is verifying data access. Approving a playbook or adding --submit would not repair those snapshots.

### Completion criterion

You have one evidence-backed explanation for the earliest blocker. Your next step tests that explanation rather than changing several independent settings at once.

## 6. Use case: inspect a candidate from idea to refusal

### Synthetic candidate

Suppose the underlying entry level is 100, stop 98, and target 103. Direction is long. A one-contract call has a quoted ask of \$0.90 and estimated fees of \$0.65. The underlying reward:risk is  $3 / 2 = 1.5$ . Modeled entry risk is \$90.65.

This arithmetic is necessary but insufficient. The option must also have trusted metadata, eligible expiry and delta, sufficient open interest, acceptable bid/ask width, and current quotes. The account must have fresh reconciled state, capacity, cash, and no entry breaker. The journal must support the structure and confirm the playbook gates.

### Walk through each layer

Star should provide a thesis tied to the actual signal, the option structure, levels, premium, and an exit plan. A score is a ranking input, not a calibrated probability of success.

Moon recomputes the level relationship and risk. If the ask were \$1.00 with the same fee estimate, the idea would exceed \$100 even though the chart setup was unchanged. If two positions or pending entries already occupy capacity, an affordable idea can still be refused.

Houston uses current broker and journal evidence. The fact that the rules layer can validate a debit spread does not make that spread executable. The journal currently refuses unsupported execution structures.

### Prompts

Star: Explain the recorded candidate's thesis, entry, stop, target, chosen structure, and quote timestamp. Separate observed fields from any inference. Do not invent missing quotes.

Moon: Using the recorded fields, explain the risk formula and each applicable rejection condition. Do not treat the scanner's score as a success probability.

If the relevant detailed idea is not exposed to chat, open the shadow report or cycle file directly. Agent prompts cannot expand the read tool's schema.

### A useful paper journal entry

Record the candidate's symbol and playbook, signal session, observation time, underlying levels, OCC contract, bid/ask, expiry, delta, total estimated risk, and final disposition. A rejection should retain its reason. A submission should retain its client\_order\_id and actual fill outcome.

### Completion criterion

You can explain why the idea advanced or stopped at each layer. You have not treated one layer's approval as an executable order or assumed that a theoretical stop determines the option's realized loss.

## 7. Use case: understand a held position and its exit

### Goal

Understand what the exit manager actually observed and why it held or exited. Use the journaled entry idea, fill inventory, and cycle's exits records. The dashboard's basis is not a current option quote.

### Inspect in this order

1. Identify the exact contract and entry session. Confirm broker and ledger inventory reconcile.
2. Check control mode and whether a submission-enabled controller is operating.
3. Read the entry idea's exit\_plan. Do not substitute a playbook summary from memory.
4. Check the option mark, available bid, underlying completed close, and any bars needed for EMA20.
5. Read the exit decision or hold note in the latest cycle. Missing inputs require a different response from a valid hold decision.
6. If there is an order, verify its status and fill evidence. Keep monitoring until the position and intent resolve.

### Synthetic premium-stop example

For a single contract with \$90.65 remaining basis, a 50% premium-stop threshold is approximately \$45.33 after cent rounding. A broker mark of \$0.40 represents \$40 per standard contract and crosses that threshold. The actual exit proceeds depend on whether a priced limit order is submitted and filled.

On the entry session, the code checks only this protective premium condition and its day-trade budget. On later sessions, an underlying stop or EMA20 rule may have priority. A premium-stop notification therefore does not establish a fixed final loss percentage.

### Prompts

Houston: Explain the latest recorded exit decision for the held contract. State the inputs, their source, the selected rule, whether an order was prepared or submitted, and what remains unknown about its fill.

Moon: Explain the priority of automatic exit rules and the entry-day exception. Keep the implemented day-trade policy separate from any claim about current broker rules.

### When the answer is "no exit inputs"

Do not assume the fallback mark was used. The controller can discard the input set after a provider exception, leaving no usable price. Inspect notifications, confirm the broker position, and follow the manual-management reference if intervention is necessary. A scheduled loop does not guarantee that each position received a valid exit evaluation.

### Completion criterion

You can distinguish a hold, a condition that fired without a price, a prepared dry decision, a submitted limit, and an actual fill. Only the last changes position exposure at the broker.

## 8. Use case: pause, leave, or recover after an interruption

### Goal

Make a deliberate operational transition without assuming that changing a local mode cancels broker activity. Begin with open positions and outstanding orders, not the desired appearance of the dashboard.

### If you only want to stop new entries

EXITS\_ONLY is the appropriate local control for continued exit management. It requires the controller to remain running with --submit and usable inputs. Confirm a subsequent cycle reads the intended mode and handles held positions. A closed browser tab is harmless to the server process; a closed owning terminal is not.

### If you want to stop the entire application

Confirm how every position and pending order will be managed first. DISABLED stops automatic entries and exits but does not undo fills or cancel broker orders. A manual flatten attempt is a new exit-order workflow and needs its own outcome verification.

### After sleep, power loss, or a process crash

1. Check the broker for current positions and order status before interpreting stale local records.
2. Confirm the old controller process is gone. A leftover controller.lock is not proof that the process is alive, but deleting it while the process runs can permit concurrent ownership.
3. Preserve the runtime evidence. Do not reset databases to make the app start cleanly.
4. If the lock is truly stale, remove only that lock as described in Volume 3.
5. Restart in a controlled diagnostic mode, reconcile, and inspect unresolved intents.
6. Resume the intended schedule only when state is understood. Do not retry an ambiguous order because the terminal disappeared.

### Prompt

Houston: Read the saved order and cycle records. Identify unresolved intents and the last confirmed outcomes. State what must be checked at the broker before resuming. Do not infer that a missing local completion means the order failed.

### Completion criterion

There is a known owner for position management, broker and local state are reconciled or the incident remains explicitly open, and no duplicate submission has been used as a recovery shortcut.

## 9. Use case: manage generative chat efficiently

### Choose the right agent and question size

Use Astra for a broad operational summary, Houston for order and reconciliation records, Moon for deterministic constraints, Star for scanner/research interpretation, and Nova for general concepts. The personalities help organize questions; they do not create different permissions.

Ask one bounded question at a time. A useful sequence is "show facts," "explain one discrepancy," then "summarize the next diagnostic." This tends to produce more checkable results than asking for a complete market outlook, trade recommendation, and account audit in one message.

### Understand the cost controls

ALPACA\_AGENTS\_LLM\_DAILY\_CAP defaults to 100 and is clamped between 0 and 2000. It counts provider calls, not messages or dollars. One answer can require multiple calls through tool rounds. The check occurs before the question's call sequence, so the setting is not a strict per-request dollar ceiling and a tool-using answer can cross the count threshold.

Usage and chat history are held in memory. Restarting the process resets that in-memory accounting. Use the provider's account controls for durable spending limits. The source default model string is claude-sonnet-4-5; current availability is a provider question, not something this guide verifies. ALPACA\_AGENTS\_LLM\_MODEL is the code's override variable.

### Save useful conclusions outside chat

Conversation history is bounded and cleared through the UI or process lifecycle. It is not the trade journal. Copy a concise factual conclusion into your operator notes with the source record and timestamp, without keys or private provider tokens. Do not assume another agent remembers the first agent's context.

### If an answer is repetitive or missing

Check whether you are in rule-based mode, whether the source reports a provider failure or budget fallback, and whether the requested tool actually exposes the record. A generic reply may be the deterministic fallback, not evidence the model reviewed the data. Return to the underlying JSON or terminal diagnostic when necessary.

### Compact prompt library

- Astra: "What changed in the last three recorded cycles? Cite times; keep unknowns explicit."
- Houston: "Which intents are still unresolved, and what is the last confirmed broker evidence?"
- Moon: "Explain why positive net P&L does not clear the cumulative-loss breaker."
- Star: "Read the QQQ summary and explain mean/median disagreement and sample limitations."
- Nova: "Explain option bid, ask, midpoint, mark, and cost basis using hypothetical numbers."

# 10. Templates for disciplined operation

## Session plan

Date and intended time window: \_\_\_\_\_

Phase: research / shadow / diagnostic / reviewed paper pilot

Code revision and runtime directory: \_\_\_\_\_

Question being tested: \_\_\_\_\_

Playbook and symbol universe: \_\_\_\_\_

Expected control mode and launch flags: \_\_\_\_\_

Known positions and unresolved intents: \_\_\_\_\_

Stop conditions and next review time: \_\_\_\_\_

## Shadow observation

Run time and completed bars session: \_\_\_\_\_

Symbols attempted / valid snapshots / errors: \_\_\_\_\_

Signal skips versus contract skips: \_\_\_\_\_

Candidate thesis, levels, contract and quote time: \_\_\_\_\_

Could I explain the result before reading the agent's prose? \_\_\_\_

Evidence filename and follow-up: \_\_\_\_\_

## Trade lifecycle review

Playbook, underlying, OCC contract and signal session: \_\_\_\_\_

Entry authorization/client order ID and decision reason: \_\_\_\_\_

Estimated entry risk / actual debit / actual fees: \_\_\_\_\_

Order submitted time / status / fill time and quantity: \_\_\_\_\_

Exit rule / inputs / chosen limit / fill outcome: \_\_\_\_\_

Net realized result and gross loss-counter effect: \_\_\_\_\_

Operational deviation, if any: \_\_\_\_\_

Lesson and whether it requires a separate experiment: \_\_\_\_\_

# 11. Weekly review and progression worksheet

## Review operational reliability first

Count planned sessions, completed sessions, successful reconciliations, scanner input failures, unexplained position/order mismatches, unknown submission outcomes, and exits lacking required inputs. Record whether each incident has a root cause and a verified resolution. A good profit result does not erase an unresolved execution defect.

## Review strategy observations second

Count clean candidates, refusals by category, actual entries, unfilled entries, actual closes, and unresolved positions. Summarize actual option P&L and known costs separately from historical underlying R. Keep separate cohorts when playbook logic, symbols, data plan, fee assumptions, or operating procedure changed.

## Write a decision in five sentences

1. This week's intended experiment was: \_\_\_\_\_
2. The reliable evidence we collected was: \_\_\_\_\_
3. The most important uncertainty or failure was: \_\_\_\_\_
4. We will continue / pause / reject / revise because: \_\_\_\_\_
5. The next review will occur after: \_\_\_\_\_

Prefer evidence-based milestones over a fixed calendar promise. A month with two trades is not equivalent to a month with thirty closed observations, and neither is enough if the records are inconsistent. A sparse strategy may require a long observation period; forcing activity changes the experiment.

## Suggested learning sequence

First, demonstrate dashboard literacy and reproducible research. Next, complete clean broker/data diagnostics and shadow sessions. Then, if justified, perform one bounded paper pilot. Finally, compare actual execution behavior with the documented assumptions. Progress only when the specific questions for the current phase have been answered.

Useful questions before expansion: Can I trace every fill to its authorization? Do actual exits match the journaled plan? Are costs complete? Do I understand unfilled and ambiguous outcomes? Is the data consistently usable? Have results been separated from operational errors? What evidence would make me stop?

Source basis: current controller, dashboard, studio, chat, scanner, rules, journal, exits, and backtest implementations under `src/alpaca_agents/`. See Volume 3 for the exact module map and external provider documentation. These worksheets are recommended operator practice, not additional application features or guarantees.